The Challenge

Team Unsupervised Predictors

Oleksandr Kudriavchenko **Volodymyr Halian**

1 Baseline Reproduction (4 points)

(a) Baseline Evaluation

The provided baseline system based on Decision Tree classifiers was successfully reproduced. The obtained Segment-based Macro F1 Score on the non-hidden test set was 0.3170. This result serves as the reference baseline for all further experiments in this project.

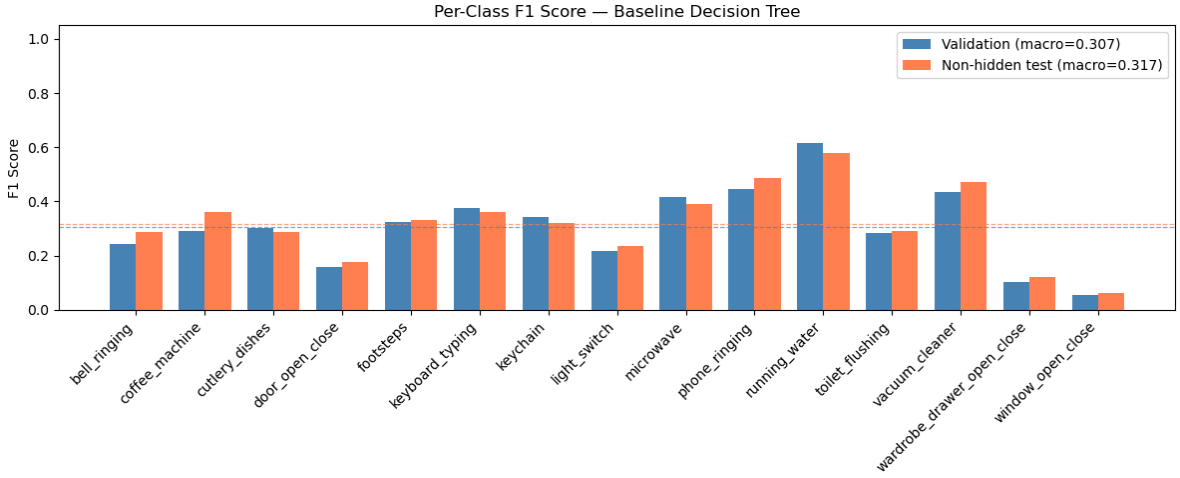


Figure 1: Per-class F1 scores of the baseline Decision Tree model on the validation and non-hidden test sets.

(b) Performance Analysis

The baseline model shows noticeable differences in performance across the sound event classes. The highest F1 score is achieved for *running_water* (0.578), followed by *phone_ringing* (0.485) and *vacuum_cleaner* (0.470). These sound events are relatively long and have distinctive acoustic characteristics, making them easier for the classifier to recognize.

In contrast, the lowest performance is observed for *window_open_close* (0.061), *wardrobe_drawer_open_close* (0.120), and *door_open_close* (0.177). These events are generally short, occur less frequently, and have similar acoustic properties.

Overall, the results indicate that longer and acoustically distinctive events are easier to classify, whereas short events remain challenging.

(c) Strengths and Limitations

The baseline approach has several advantages. It is computationally efficient, easy to train, and simple to interpret. Training independent Decision Tree classifiers for each class provides a straightforward baseline that can be reproduced quickly.

However, the baseline also has several important limitations. Decision Trees are limited in their ability to learn complex decision boundaries. Furthermore, each sound class is predicted independently, meaning that relationships between simultaneously occurring sound events are ignored. The temporal resolution of one second also limits the detection of short events, while the baseline does not explicitly model temporal dependencies between neighboring segments. These design choices explain why the model performs well on long, characteristic sounds but struggles with short or acoustically similar events.

2 Simple Classifiers (10 points)

2.1 (a) Previous classifier

In the previous project stage we stopped on a Feed-forward Neural Network. It had four layers with 128, 256, 128 and 32 neurons. The model was trained for 30 epochs with learning rate 0.01 and dropout 0.3. In comparison with a random classifier and k-nearest neighbors classifier it achieved the highest performance. That was the reason, why we have choosed it.

2.2 (b) Hyperparameter tuning

Now we want to adapt the FNN classifier to a given pipeline. As data was preprocessed for us differently, compared as we did it before, we decided to use 512, 256, 128 and 64 neurons on the four layers, as the number of input parameters is much higher than before. Learning rate and dropout stayed the same, but the number of epochs was increased from 30 to 50, but only the best model (F1-wise) was saved.

In the second experiment we decided to reduce number of layers by one, deleting the 512 neuron layer. The number of trainable parameters decreased, so we also turned the dropout off.

In the Table 1 we compared Precision(P), Recall(R) and F1 scores for both models. Some of the values decreased, but in general the second classifier with 3 layers works better. This shows us that for better results not always bigger model is required.

Annotation	**P (NN 4L, 0.3D)**	**R (NN 4L, 0.3D)**	**F1 (NN 4L, 0.3D)**	**P (NN 3L, 0.0D)**	**R (NN 3L, 0.0D)**	**F1 (NN 3L, 0.0D)**
bell_ringing	0.120	0.836	0.210	0.204	0.618	0.307
coffee_machine	0.175	0.737	0.283	0.225	0.743	0.345
cutlery_dishes	0.269	0.803	0.403	0.387	0.711	0.501
door_open_close	0.230	0.776	0.355	0.277	0.715	0.400
footsteps	0.398	0.768	0.524	0.464	0.686	0.554
keyboard_typing	0.391	0.764	0.517	0.424	0.783	0.550
keychain	0.320	0.766	0.452	0.322	0.746	0.449
light_switch	0.135	0.772	0.230	0.149	0.741	0.248
microwave	0.247	0.798	0.377	0.385	0.681	0.492
phone_ringing	0.350	0.724	0.472	0.505	0.649	0.568
running_water	0.696	0.754	0.724	0.595	0.824	0.691
toilet_flushing	0.245	0.735	0.367	0.149	0.848	0.253
vacuum_cleaner	0.414	0.839	0.554	0.422	0.823	0.558
wardrobe_drawer_open_close	0.110	0.721	0.190	0.119	0.723	0.205
window_open_close	0.067	0.756	0.123	0.078	0.725	0.141
Macro F1		0.3854			0.4175	

Table 1: Comparison of FNN (4 layers, dropout 0.3) vs FNN (3 layers, dropout 0)

2.3 (c) Decision tree vs FNN

Now we would like to compare the performance of a baseline classifier(decision trees) and our model. Table 2 represents the comparison of those in terms of precision, Recall and F1-scores. In general, the FNN model has higher Macro-F1 score, that shows us improved balance between Precision and Recall.

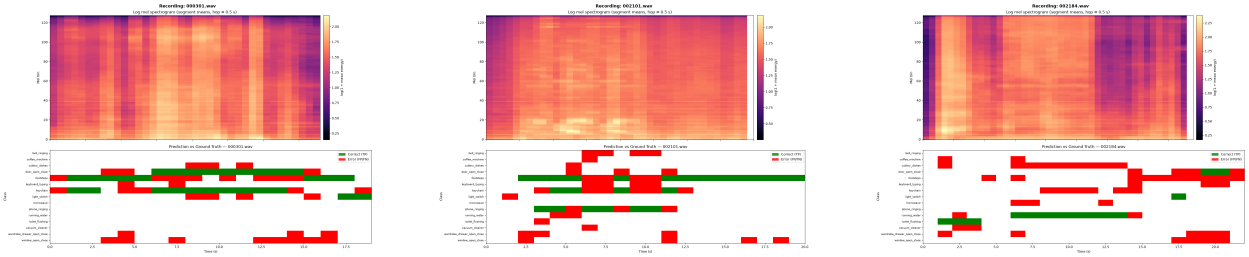
Annotation	P (Tree)	R (Tree)	F1 (Tree)	P (FNN)	R (FNN)	F1 (FNN)
bell_ringing	0.426	0.218	0.288	0.204	0.618	0.307
coffee_machine	0.504	0.283	0.362	0.225	0.743	0.345
cutlery_dishes	0.294	0.281	0.287	0.387	0.711	0.501
door_open_close	0.200	0.159	0.177	0.277	0.715	0.400
footsteps	0.384	0.293	0.333	0.464	0.686	0.554
keyboard_typing	0.372	0.347	0.359	0.424	0.783	0.550
keychain	0.365	0.287	0.321	0.322	0.746	0.449
light_switch	0.240	0.228	0.234	0.149	0.741	0.248
microwave	0.408	0.373	0.390	0.385	0.681	0.492
phone_ringing	0.534	0.444	0.485	0.505	0.649	0.568
running_water	0.655	0.517	0.578	0.595	0.824	0.691
toilet_flushing	0.288	0.292	0.290	0.149	0.848	0.253
vacuum_cleaner	0.485	0.455	0.470	0.422	0.823	0.558
wardrobe_drawer_open_close	0.134	0.109	0.120	0.119	0.723	0.205
window_open_close	0.057	0.066	0.061	0.078	0.725	0.141
Macro F1		0.3170			0.4175	

Table 2: Comparison of Tree model vs improved model performance on event detection

2.4 (d) Error analysis

After choosing the best so far model, we need to understand, where do we get errors (as the scores are not 100%). Figure 2 depicts three sound recordings, which are represented by mel spectrograms and class predictions. Green blocks on predictions show correct predictions, and red ones show false predictions.

We see that our model predicts the correct classes, but with this it also detects sound that arent active in respective moments. Two of the recordings have false positive for a coffee_machine class. It may indicate, that there was not enough of recordings with coffee machines, so in the train data, the class wasn't represented enough. Also there are False positives in wardrobe_door_open_close, but this may have a different reason. This is a low frequency sound, that may be misclassified with something else, because of the model imperfections. Still, the true positive results are quite good, and may be improved during post-processing.



(a) First recording (b) Second recording (c) Third recording

Figure 2: Demonstration of three recordings (green correctly predicted)

3 Post-Processing and Temporal Refinement (8 points)

3.1 (a) Implement one of the two post-processing techniques and investigate the effect of at least one important parameter

We implemented temporal smoothing as a post-processing step. Since the baseline classifier predicts each one-second segment independently, the predictions can be temporally noisy and fragmented. To reduce this effect, neighboring predictions of the same class were merged if the temporal gap between them was smaller than or equal to a smoothing parameter α.

We investigated three values of the smoothing parameter: $\alpha = 0.5$, $\alpha = 1.0$, and $\alpha = 2.0$. A larger value of α allows events of the same class to be merged across longer temporal gaps, producing smoother prediction trajectories.

3.2 (b) Compare the performance of the post-processed system with the corresponding system without post-processing

The effect of temporal smoothing was evaluated on the non-hidden test set using the Segment-based Macro F1 Score.

Method	Macro F1
No post-processing	0.3170
Temporal smoothing, $\alpha = 0.5$	0.3170
Temporal smoothing, $\alpha = 1.0$	0.3389
Temporal smoothing, $\alpha = 2.0$	0.3516

Table 3: Effect of temporal smoothing on the non-hidden test set.

Temporal smoothing improved the Macro F1 Score from 0.3170 to 0.3516. The best result was with $\alpha = 2.0$. This suggests that the baseline predictions were temporally fragmented, and merging nearby predictions helped to create more consistent event regions.

The improvement is likely strongest for longer events such as *running_water*, *vacuum_cleaner*, or *phone_ringing*, where small gaps between consecutive detections should usually belong to the same sound event. However, too much smoothing could be bad for very short events, because it may merge separate events or extend predictions beyond the true event duration.

4 Reflection and Real-World Considerations (5 points)

4.1 (a) Technical Limitations

Our current best system achieved a Segment-based Macro F1 Score of 0.4175 on the non-hidden test set. Although this is a clear improvement compared to the original Decision Tree baseline, the model still has several technical limitations.

First, the system still struggles with short sound events, such as *window_open_close*, *door_open_close*, and *wardrobe_drawer_open_close*. These events are difficult because they often last for a very short time and may only occupy a small part of a one-second segment. This could be improved by using a finer temporal resolution, more precise onset and offset estimation, or sequence models that can better capture short temporal patterns.

Second, the model can confuse similar household sounds. For example, doors, drawers, windows, and light switches can produce similar clicking sounds. This limitation could be addressed by using richer feature representations, combining multiple feature types or using better architecture that can learn more complex patterns.

Third, the model still produces false positives and false negatives in some recordings. This indicates that it does not always distinguish between different sound events correctly. Future improvements could focus on better model architectures, hyperparameter tuning, and training on more diverse data.

Finally, the model may still perform worse on rare classes because there are fewer training examples available. Future work could address this by collecting more balanced data, using data augmentation or applying class-weighted losses during training.

4.2 (b) Real-World Deployment

In its current state, the system is useful as a prototype, but it is not yet reliable enough for deployment in a real-world smart-home assistant. A Macro F1 Score of 0.4175 shows that the system can detect some sound events reasonably well, but the performance is still too low for an application.

From a practical perspective, the system should be fast enough to use because it uses audio features that are prepared before training and does not require a very large model. This means that the computational cost and delay would probably be acceptable. However, the main problem is robustness. Real homes are noisy and unpredictable. There can be background noise, several sounds happening at the same time, different microphones, different rooms, and sounds that were not included in the training data.

Reliability is also very important for a smart-home assistant. If the system produces too many false positives, users may receive unnecessary notifications. If it misses important sounds, users may lose trust in the system.

Overall, the current system is a reasonable experimental sound event detection prototype, but additional improvements are necessary before it could be used in a real smart-home product.

5 Bonus 1 (up to 5 points):

5.1 (a) CRNN model

For better performance we decided to create a Convolutional Recurrent Neural Network. We have used one convolutional layer with 32 output channels , one GRU layer with 64 neurons and one output linear layer. Dropout value is 0.3 . After 5 epochs of training the Macro-F1 score was 0.36, that is a little bit better, than the baseline, but still we believe, that this architecture has more potential.

The advantage of this model that it analyses the raw audio, instead of preprocessed sound parameters.

5.2 (b) Improvement strategies

First of all, we would change the architecture a by adding more convolutional layers. Bigger model can capture more interesting patterns, That may be useful in our usecase, when we have 15 classes and polyphony.

We decided to also tweak the hyperparameters. After a few experiments the model became heavier: it has three convolutional layers with 32, 64, 64 output channels, a GRU layer with 128 neurons and a linear output layer. The dropout stayed the same. The number of epochs was increased to 50, but only the best model was saved (determined by the F1-score).

5.3 (c) Performance comparison

The performance after enhancements became greater! Now the Macro-F1 score is 0.6! Figure 3 demonstrates spectrogram and class labels for an audio file, similarly, as we did in Section 2. We see, that now we much have random False positives. The predictions are much cleaner.

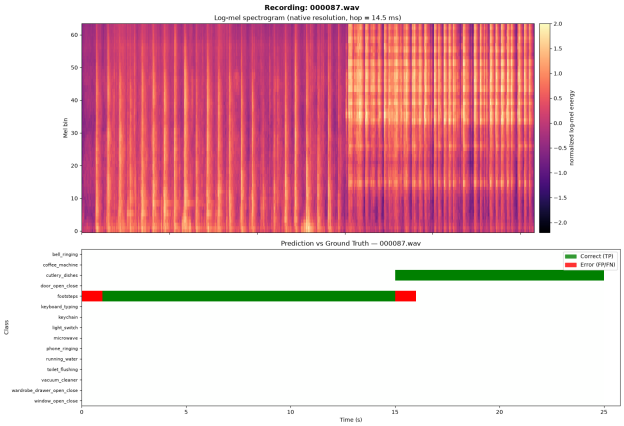


Figure 3: Demonstration of three recordings (green correctly predicted)

In Table 4 the performance metrics for 3 models from this report are shown. We see, that CRNN is better than our previous FNN classifier, which is also better than Decision tree classifier.

Annotation	**Decision Tree**			**FNN**			**CRNN**		
	P	R	F1	P	R	F1	P	R	F1
bell_ringing	0.426	0.218	0.288	0.204	0.618	0.307	0.290	0.400	0.340
coffee_machine	0.504	0.283	0.362	0.225	0.743	0.345	0.550	0.680	0.610
cutlery_dishes	0.294	0.281	0.287	0.387	0.711	0.501	0.460	0.850	0.600
door_open_close	0.200	0.159	0.177	0.277	0.715	0.400	0.430	0.820	0.560
footsteps	0.384	0.293	0.333	0.464	0.686	0.554	0.510	0.820	0.630
keyboard_typing	0.372	0.347	0.359	0.424	0.783	0.550	0.750	0.910	0.820
keychain	0.365	0.287	0.321	0.322	0.746	0.449	0.590	0.860	0.700
light_switch	0.240	0.228	0.234	0.149	0.741	0.248	0.200	0.680	0.310
microwave	0.408	0.373	0.390	0.385	0.681	0.492	0.660	0.730	0.700
phone_ringing	0.534	0.444	0.485	0.505	0.649	0.568	0.660	0.810	0.730
running_water	0.655	0.517	0.578	0.595	0.824	0.691	0.840	0.880	0.860
toilet_flushing	0.288	0.292	0.290	0.149	0.848	0.253	0.560	0.790	0.650
vacuum_cleaner	0.485	0.455	0.470	0.422	0.823	0.558	0.760	0.800	0.780
wardrobe_drawer_open_close	0.134	0.109	0.120	0.119	0.723	0.205	0.260	0.610	0.370
window_open_close	0.057	0.066	0.061	0.078	0.725	0.141	0.220	0.690	0.330
Macro F1		0.3170			0.4175			0.6000	

Table 4: Comparison of Decision Tree, FNN, and CRNN models

6 Bonus 2 (up to 5 points):

For the second bonus task, we fine-tuned a pre-trained sound event detection model on the provided raw waveform dataset. We used the BEATs Strong checkpoint, which was pre-trained on AudioSet Strong, as the starting point. The model was adapted to predict the 15 target sound event classes of the challenge dataset. After fine-tuning, the predictions were converted into the required event format with filename, annotation, onset, and offset columns. The final system was evaluated using the same Segment-based Macro F1 Score as in the previous sections.

System	Segment-based Macro F1
Original Decision Tree baseline	0.3170
Best classical / MLP-based system	0.4175
Best deep-learning system from Bonus 1	0.6000
Fine-tuned BEATs Strong model	0.6420

Table 5: Comparison of the pre-trained BEATs Strong model with previous systems.

The fine-tuned BEATs Strong model achieved the best overall performance with a Segment-based Macro F1 Score of 0.6420. This improved over the previous best deep-learning system, which achieved 0.6000, and clearly outperformed both the original Decision Tree baseline and the best classical model.

The improvement is likely due to the stronger audio representation learned during large-scale pre-training. Since BEATs was pre-trained on a large audio dataset, it can capture more complex spectral and temporal patterns than models trained only from scratch on the challenge data. This is especially useful for sound events with characteristic acoustic structure, such as *running_water*, *vacuum_cleaner*, and *phone_ringing*.

However, the model still does not solve all problems completely. Short and transient events such as *door_open_close*, *window_open_close*, and *wardrobe_drawer_open_close* remain challenging because their acoustic patterns are brief and sometimes similar to each other. Therefore, even with a stronger pre-trained model, precise temporal localization and separation of similar short sounds remain difficult.

Overall, the results show that pre-training provides a clear advantage for this sound event detection task. The BEATs Strong model achieved the best performance among all tested systems and demonstrates that using large-scale pre-trained audio representations can improve sound event detection compared to both classical machine learning models and smaller deep-learning systems trained from scratch.

7 Disclosure of LLM and AI Tool Use

LLM's were used while preparing this report for fast access to python code examples (like graph generation), broader understanding of the concepts or grammar correction. Verification was done logically in case of code snippets, or checked in with lecture slides in case of understanding the meaning of new terms.Additionally, we used LLM to generalize and summarize some parts of the analysis to avoid unnecessary verbosity.